SafeKeep Support Agent

Software Functional Specification

Sai Varshini Thupakula

Contents

1. Introduction....3
2. Requirements catalog....4
3. Assumptions and design decisions....7
4. Architecture....9
5. Domain model....13
6. Investigation state machine....14
7. Event intake and deduplication....15
8. Agent loop and LLM contract....17
9. Tools, permissions and approval gate....20
10. Applications and scenarios....23
11. REST API....26
12. User interface....27
13. Error handling and reliability....28
14. Security....29
15. Out of scope and future work....29
16. Revision history....30

1. Introduction

1.1 Purpose

This document specifies the functional behavior, architecture, data model, interfaces and rules of the SafeKeep Support Agent. It translates the PRD “Event-Driven Application Support Agent” into a buildable and testable design. It is the input for the implementation (Problem 2) and the unit-test plan (Problem 3). Every requirement carries an ID, and every later artifact (code, tests, traceability matrix) refers back to those IDs.

1.2 Product context

The Agent is demonstrated against **SafeKeep**, a fictional enterprise VM backup and restore platform modeled on real-world experience with such products. Customers register datasources (for example, a VMware vCenter), define backup policies, and schedule backup and report jobs. The platform's main components are:

Table 1. SafeKeep platform components

Component	Role in the platform	Role for the Agent
Backup Server	Management VM: UI, datasources, policies, job scheduling, job sessions, reporting	Monitored application #1
Backup Storage Server	Stores backup data in disk pools; expires old backups; replicates to a DR site	Monitored application #2
Data Mover Proxy	Reads VM disk data through VMware's data-protection APIs and writes it to storage	Reached through Backup Server tools only
vCenter	VMware manager where customer VMs live (a datasource)	External dependency checked by tools

1.3 Scope

In scope is a working, demo-able vertical slice that covers every PRD requirement at demo fidelity:

- Event intake from two simulated applications, with validation, duplicate linking and recurrence detection.
- An LLM tool-calling agent (local model via Ollama) that gathers evidence, separates confirmed facts from suspected causes, and recommends next steps.
- An approval gate for corrective actions, verification of action outcomes, and a full audit trail.
- A web UI for investigations, follow-up questions, feedback, history, administration and health.

Section 15 lists what is explicitly out of scope.

1.4 Conventions

Table 2. Identifier conventions

Prefix	Meaning	Example
FR-xxx-nn	Functional requirement: what the system does	FR-ACT-03
NFR-xxx-nn	Non-functional requirement: how well it does it	NFR-SEC-02

Prefix	Meaning	Example
UC-n	PRD use case (end-to-end scenario family)	UC-2
MET-nn	PRD success measure	MET-03
Dn	Design decision (Section 3)	D5
An	Assumption (Section 3)	A1
S-xx	Seeded demo scenario (Section 10)	S-1A

1.5 Glossary

Table 3. Glossary

Term	Definition
Application	A system that sends events to the Agent and exposes tools to it (here: Backup Server, Storage Server).
Event	A notification from an application that something needs attention (e.g. HIGH_MEMORY).
Investigation	The Agent's record of one issue: state, evidence, findings, actions, conversation. The equivalent of a ticket.
Correlation key	application + event type + resource. Two events with the same key describe the same issue.
Tool	A named, read-only diagnostic operation an application exposes (e.g. get_metrics).
Action	A named operation that changes application state (e.g. purge_stale_sessions). Requires approval.
Evidence	A recorded tool call and its result, identified as E1, E2, …
Finding	A statement produced by the Agent, labeled CONFIRMED, SUSPECTED or RULED_OUT, citing evidence.
Adapter	Application-specific code that implements an application's tools and actions.
State	Progress of the Agent's work on an investigation.
Resolution	Whether the underlying problem is fixed. Independent of state.

2. Requirements catalog

Each PRD statement is broken down into an atomic requirement with a unique ID. The **Section** column points to the part of this specification that defines how the requirement is met. Together with the test cases in the Unit-Test Plan, this forms the requirement-to-test traceability matrix.

2.1 Event intake (EVT)

Table 4. Event intake requirements

ID	Requirement	PRD	Section
FR-EVT-01	Accept events only from configured applications and configured event types; reject others with a clear error.	7.1	7
FR-EVT-02	Identify application, event type, affected resource or operation, and details; validate required fields.	6.1, 10	7.2
FR-EVT-03	Acknowledge events promptly; run the investigation asynchronously.	NFR	4.6

ID	Requirement	PRD	Section
FR-EVT-04	Link duplicate or repeated events about the same issue to the existing investigation.	6.2, 7.1	7.3–7.5
FR-EVT-05	Report event intake failures.	7.1	7.6

2.2 Investigation lifecycle (INV)

Table 5. Investigation requirements

ID	Requirement	PRD	Section
FR-INV-01	Create an investigation for each new, non-duplicate event.	6.2	7
FR-INV-02	Track states RECEIVED, INVESTIGATING, WAITING_FOR_INFO, COMPLETED, UNABLE_TO_COMPLETE; allow only valid transitions.	7.1	6
FR-INV-03	Show investigation progress while it runs.	6.5	8, 12
FR-INV-04	On failure, end in UNABLE_TO_COMPLETE with a stated reason and keep the user informed.	7.1	13

2.3 Data gathering (DATA)

Table 6. Data gathering requirements

ID	Requirement	PRD	Section
FR-DATA-01	Retrieve metrics, process/service details, logs, job information, settings and troubleshooting guidance through tools.	7.2	10.3
FR-DATA-02	Use only sources configured and permitted for the application.	6.3	9.2
FR-DATA-03	Store evidence references and tool results sufficient to explain each finding.	7.2	5, 8.3
FR-DATA-04	Request more information from the user when evidence is insufficient.	6.4	8.7

2.4 AI analysis (AI)

Table 7. AI analysis requirements

ID	Requirement	PRD	Section
FR-AI-01	Use an LLM to analyze the event and the evidence.	7.2	8
FR-AI-02	The LLM chooses tools only from those configured for that application.	7.2	8.2, 9.2
FR-AI-03	Distinguish confirmed measurements from suspected causes; never present a suspicion as fact.	5.1, 7.2	8.5
FR-AI-04	When evidence is incomplete, state what is missing and what to check next.	7.2	8.4–8.5

2.5 Use cases (UC)

Table 8. Use cases

ID	Requirement	PRD	Section
UC-1	High resource usage: check resource data, rank top consumers, correlate with logs, service status and recent changes; report leading consumers with time range, evidence and next checks.	5.1	10.5

ID	Requirement	PRD	Section
UC-2	Failed job: retrieve job details, logs, settings and service status; summarize the error, explain the likely cause when supported, recommend troubleshooting steps.	5.2	10.5
UC-3	Dependency unavailable: check service status, error logs and affected operations; explain impacted functions; suggest next checks or escalation.	5.3	10.5

2.6 User interaction (USR)

Table 9. User interaction requirements

ID	Requirement	PRD	Section
FR-USR-01	Present event summary, findings, evidence, recommended actions and resolution status in clear language.	7.3	12.4
FR-USR-02	Let users view progress.	7.3	12.4
FR-USR-03	Let users provide context.	7.3	8.7
FR-USR-04	Let users ask follow-up questions.	7.3	8.7
FR-USR-05	Let users request more investigation.	7.3	8.7
FR-USR-06	Let users give feedback on whether findings were useful.	7.3	12.4

2.7 Tools, permissions and actions (ACT)

Table 10. Tool and action requirements

ID	Requirement	PRD	Section
FR-ACT-01	Administrators configure which tools and operations are available per application.	7.4	10.2, 12.5
FR-ACT-02	Validate every tool request and check permissions before execution.	7.4	9.2
FR-ACT-03	Require explicit approval before any action that changes application state.	6.6, 7.4	9.4
FR-ACT-04	Execute only the exact action approved, with the exact parameters.	7.4	9.4
FR-ACT-05	Verify and report the action result; never report a failed action as successful.	7.4	9.5
FR-ACT-06	If an action is unavailable or not permitted, give instructions instead.	7.4	8.5
FR-ACT-07	Record tool requests, results, approvals and action outcomes.	6.7, 7.4	5, 14

2.8 Configuration and history (CFG)

Table 11. Configuration requirements

ID	Requirement	PRD	Section
FR-CFG-01	Administrators configure application connections, event types, data access, tools, permissions and guidance.	7.5	10.2
FR-CFG-02	Authorized users retrieve past investigations with evidence and actions.	7.5	12.3
FR-CFG-03	Show Agent health, connection status and integration errors.	7.5	12.6
FR-CFG-04	Support new applications without redesigning the shared Agent.	4, 7.5	10.7

2.9 Non-functional requirements (NFR)

Table 12. Non-functional requirements

ID	Requirement	Section
NFR-SEC-01	Protect credentials and application data; secrets never logged, displayed or sent to the LLM.	14
NFR-SEC-02	Enforce user access and tool permissions on the server.	9.3, 14
NFR-REL-01	Handle application, tool and AI-service failures gracefully and show status.	13
NFR-RESP-01	Acknowledge events promptly; show progress during long investigations.	4.6, 12.4
NFR-SCAL-01	Run multiple investigations concurrently without mixing their data.	4.6, 13.2
NFR-MAINT-01	Keep application-specific integrations separate from shared Agent behavior.	4.3, 10.7
NFR-USAB-01	Clear findings, actionable errors, understandable approval requests.	12

2.10 Success measures (MET)

Table 13. Success measures

ID	Measure	Section
MET-01	Median time from event receipt to first recommended next step.	12.6
MET-02	Share of investigations with supporting evidence, and share with positive feedback.	12.6
MET-03	Success rate of approved actions; count of actions executed without approval (target: zero).	12.6

3. Assumptions and design decisions

The PRD leaves several points open. Each was resolved explicitly; the decisions below are binding for implementation and testing.

3.1 Design decisions

Table 14. Design decisions

ID	Topic	Decision	Rationale
D1	Duplicate events	Correlation key = application + event type + resource. Link to an open investigation with the same key; otherwise create a new one and mark it a recurrence of the most recent closed one. Ignore repeated eventIds (idempotency). Create-or-link is race-safe.	Deterministic and testable. LLM-based matching rejected: slow, non-repeatable.
D2	Users and roles	Spring Security with three demo users: alice (APP_USER), sam (SUPPORT_ENGINEER), adam (ADMIN). Applications authenticate with per-application API keys.	Makes access enforcement real and demonstrable; separates human and machine identity.
D3	Who approves	Each action declares requiredRole and risk in configuration (default SUPPORT_ENGINEER).	PRD: administrators control actions. Risk-proportionate control.

ID	Topic	Decision	Rationale
D4	State vs resolution	Two independent fields: state (Agent's work) and resolution (UNRESOLVED, PARTIALLY_RESOLVED, RESOLVED, UNKNOWN).	An investigation can be complete while the problem persists.
D5	Confirmed vs suspected	LLM must cite evidence IDs; code downgrades CONFIRMED findings without valid evidence to SUSPECTED.	Deterministic guardrail over AI output.
D6	Waiting for info	The Agent may ask the user one question; state becomes WAITING_FOR_INFO until the user replies.	Implements PRD 6.4 with the user as information source.
D7	Follow-ups	A question gets an answer without a state change. Added context or “investigate more” reopens the investigation.	Keeps COMPLETED meaningful.
D8	Action verification	After execution the system re-checks with a configured read tool and sets resolution from the result.	FR-ACT-05: outcome is measured, not assumed.
D9	Thresholds	Applications decide threshold breaches; configured thresholds are shown as context.	PRD section 10.
D10	Guidance	Markdown runbooks per application, keyword search tool.	Sufficient for demo; vector search is future work.
D11	LLM failure	Two retries with backoff, then UNABLE_TO_COMPLETE (“AI service unavailable”); evidence remains visible.	NFR-REL-01.
D12	Loop cap	Maximum 8 tool calls per run; then a partial result listing missing evidence.	Prevents runaway loops and bounds latency.
D13	Configuration	One YAML file per application loaded at startup; admin can toggle tools and actions at runtime.	Config-driven applications (FR-CFG-04).
D14	LLM provider	Ollama only, behind an LlmClient interface; FakeLlmClient used for unit tests.	Local, free, offline; repeatable tests.
D15	Model	qwen2.5:3b. qwen3:4b rejected: its reasoning mode could not be disabled (118 s per tool call). llama3.2:3b was fast (1.7 s) but stopped after one tool call and ignored evidence. qwen2.5:3b produced evidence-backed CONFIRMED findings and the correct proposal.	Measured on the target laptop (M1, 8 GB) with scenario S-1A.
D16	Framework version	Spring Boot 4.1.2 instead of 3.x.	Spring Boot 3.5 reached open-source end of life in June 2026.
D17	Agent guards for a small model	Code nudges the model when it stops before 4 tool calls (max 2 nudges), skips identical repeated calls, and sends it back once to use unused tools before asking the user a question.	Observed in test runs: early stopping, repeated calls, asking the user for data a tool could provide.
D18	Two-phase answer	Tool phase first; then a separate final request in JSON mode, without tools, that re-lists every evidence item and asks for observations of abnormal values before findings.	The model ignored evidence it had already gathered when writing its conclusion.
D19	Proposals only for present problems	Rule R-V8: a proposed action is kept only when the resolution is UNRESOLVED.	A re-investigation proposed a fix for a problem that was already gone.
D20	API clients and CSRF	Browser sessions use form login with CSRF protection. API clients use HTTP Basic or the API key; requests carrying an Authorization header are exempt from CSRF.	CSRF attacks rely on cookies; a malicious page cannot add an Authorization header.

ID	Topic	Decision	Rationale
D21	Context window	Kept at Ollama's default (4,096 tokens). Logs showed the largest request at 2,223 tokens and no truncation.	A suspected cause of weak reasoning was checked against evidence and rejected.

3.2 Assumptions

Table 15. Assumptions

ID	Assumption
A1	Applications are simulated in-process behind the AppAdapter interface. Replacing an adapter with a real HTTP integration does not change the Agent core.
A2	Events contain enough detail to identify the event type and affected resource (PRD section 10).
A3	Single-node deployment. Concurrency control is in-process; distributed locking is out of scope.
A4	Roles are global: every user can see all applications. Per-application access is future work.
A5	Seed data contains raw observations only, never the root cause. The Agent must infer conclusions.
A6	LLM output is non-deterministic. Correctness rules are enforced by code, and unit tests use a fake LLM.
A7	Scenario timestamps are relative to the moment the event is sent, so demos look current.
A8	All names (product, hosts, customers) are fictional. No real customer or vendor data is used.
A9	Admin tool and action toggles are held in memory and reset on restart. Investigations, evidence, proposals and audit are stored in the H2 database file and survive restarts.
A10	In the demo, events may carry an X-Demo-Scenario header that selects the seed data set. A real integration would not need it.

4. Architecture

4.1 System context

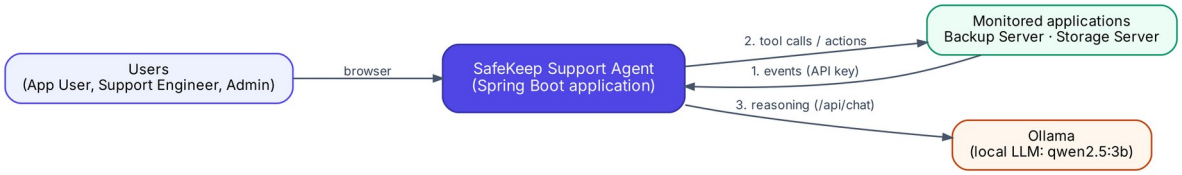


Figure 1. System context

The Agent is a single Spring Boot application. Users reach it through a browser. Monitored applications send events to it and expose tools to it; in this build they are simulated inside the same process. All reasoning is done by a local LLM served by Ollama.

4.2 Component architecture

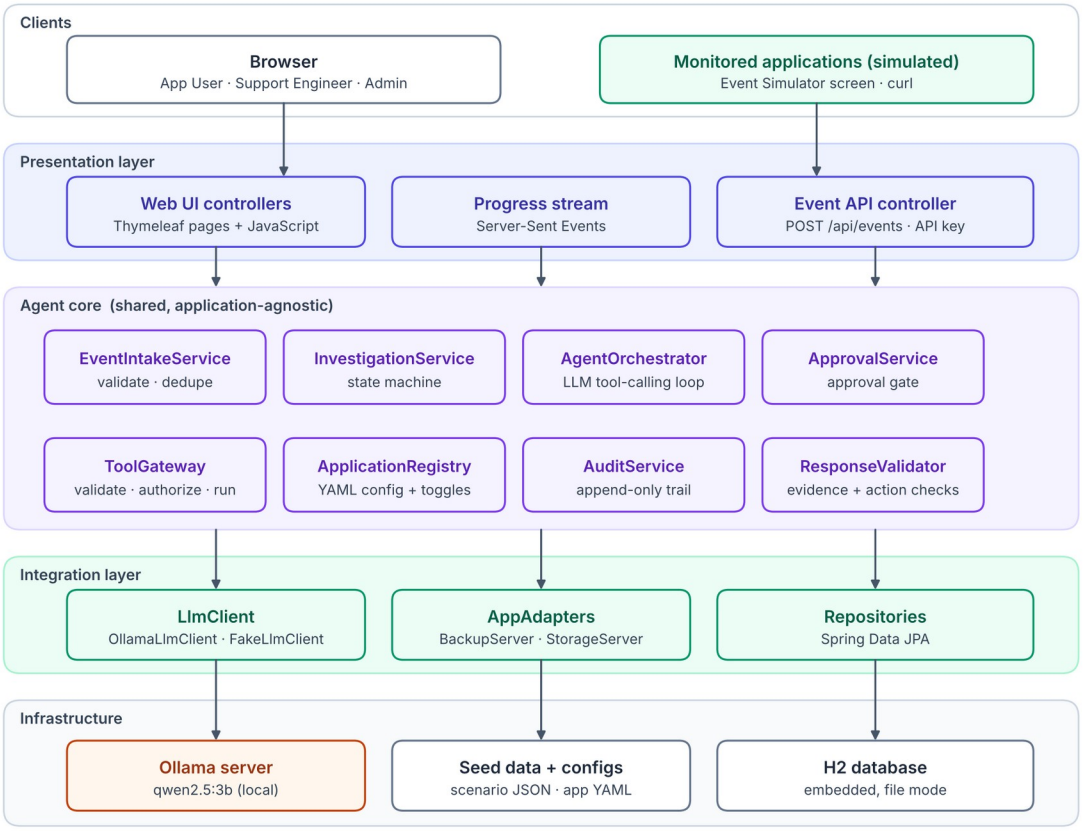


Figure 2. Layered component architecture

Table 16. Component responsibilities

Component	Responsibility	Requirements
Event API controller	Authenticates the application (API key), parses the event, returns 202/200/4xx.	FR-EVT-01, -02
EventIntakeService	Validation, idempotency, correlation, create-or-link, recurrence linking.	FR-EVT-01…05, FR-INV-01
InvestigationService	Owns the state machine and resolution status; publishes progress events.	FR-INV-02…04
AgentOrchestrator	Runs the LLM tool-calling loop; handles follow-ups and re-investigation.	FR-AI-01…04, FR-DATA-04
ToolGateway	Single entry point for every tool and action execution: existence, enablement, argument validation, timeout, recording.	FR-ACT-02, FR-DATA-02
ResponseValidator	Validates the LLM's final output: schema, evidence IDs, action rules, downgrades.	FR-AI-03, FR-ACT-06
ApprovalService	Approval gate: role check, single use, expiry, exact-parameter execution, verification.	FR-ACT-03…05
AuditService	Append-only record of every significant step.	FR-ACT-07
ApplicationRegistry	Loads application YAML; holds runtime enable/disable toggles.	FR-CFG-01, FR-ACT-01

Component	Responsibility	Requirements
LlmClient	Interface to the model. OllamaLlmClient (runtime), FakeLlmClient (tests).	D14
AsyncRunScheduler	Bounded worker pool; retries work rejected by a full queue.	FR-EVT-03
AppAdapters	Application-specific tool and action implementations backed by seed data.	NFR-MAINT-01
Progress stream	Server-Sent Events channel per investigation for live progress.	FR-INV-03

4.3 Design principles

Table 17. Design principles

#	Principle	Consequence
P1	The LLM reasons; code enforces.	Permissions, approvals, state transitions, evidence checks and outcome reporting are deterministic code. The LLM only chooses tools and writes analysis.
P2	Application specifics live at the edge.	Everything about Backup Server or Storage Server is in its YAML, adapter and seed data. The core never names a specific application.
P3	Everything is recorded.	Each tool call becomes evidence; each decision becomes an audit entry. Findings are traceable to data.
P4	Fail visibly, never silently.	Every failure ends in an explicit status with a reason shown to the user.
P5	Least privilege for the model.	The LLM can call read-only tools only. Actions can only be proposed, and only a human can approve them.

4.4 Technology stack

Table 18. Technology stack

Concern	Choice	Reason
Language / runtime	Java 21	Candidate's primary language; records and modern features.
Framework	Spring Boot 4.1.2 (Web MVC, Security, Data JPA)	Industry standard; one-command run (D16).
UI	Thymeleaf + small JavaScript file + one CSS file	Server-rendered, no Node toolchain; actions via fetch, live updates via Server-Sent Events.
Database	H2 (file mode)	Embedded, zero installation.
LLM	Ollama + qwen2.5:3b	Local, free, tool-calling support; fits 8 GB RAM (D15).
HTTP client	Spring RestClient	Calls Ollama /api/chat.
Configuration	YAML (Jackson)	Readable per-application config.
Testing	JUnit 5, Mockito, AssertJ	Standard Java unit-test stack.
Build	Maven	Standard; `mvn spring-boot:run`.

4.5 Package structure

```
com.safekeep.agent
├── api            REST + SSE controllers, DTOs, error handler
```

```
├── web            Thymeleaf page controllers
├── intake         EventIntakeService, CorrelationKey, IdempotencyStore
├── investigation  Investigation entity, state machine, InvestigationService
├── agent          AgentOrchestrator, PromptBuilder, ResponseValidator
├── llm            LlmClient, OllamaLlmClient, FakeLlmClient
├── tools          ToolGateway, ToolDefinition, ArgumentValidator
├── approval       ActionProposal, ApprovalService, ActionVerifier
├── apps           ApplicationRegistry, AppAdapter
│   ├── backupserver   BackupServerAdapter + scenario data
│   └── storageserver  StorageServerAdapter + scenario data
├── audit          AuditEntry, AuditService
├── security       SecurityConfig, ApiKeyFilter, demo users
└── health         HealthService, MetricsService
```

4.6 Concurrency model

- `POST /api/events` performs only validation and create-or-link, never an LLM or tool call, and returns in under 200 ms. The investigation runs on a bounded thread pool (4 workers, queue 50) after the transaction commits (FR-EVT-03).
- At most one worker processes a given investigation at a time (per-investigation lock). Different investigations run in parallel.
- If the pool and its queue are full, the investigation stays RECEIVED and is retried every 15 seconds; nothing is lost.
- The orchestrator keeps all run state in a per-run context object; no mutable state is shared between investigations (NFR-SCAL-01).
- Create-or-link is serialized per correlation key, so two identical events arriving together produce one investigation (D1).

5. Domain model

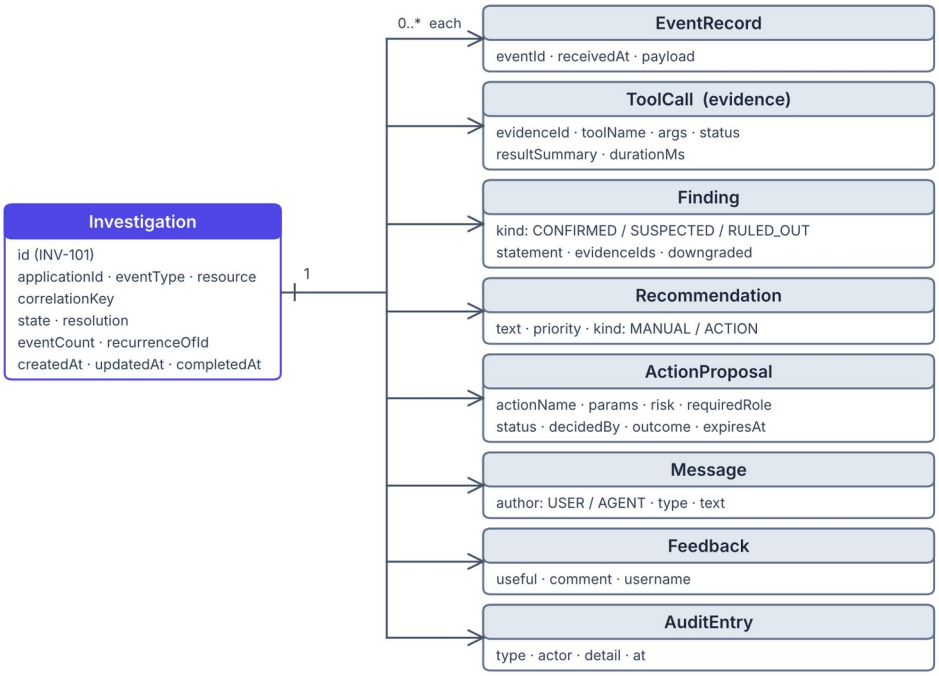


Figure 3. Domain model. Investigation is the aggregate root.

5.1 Investigation

Table 19. Investigation fields

Field	Type	Description
id	String	Human-readable ID, e.g. INV-101.
applicationId	String	e.g. backup-server.
eventType	String	e.g. HIGH_MEMORY.
resource	String	Affected resource or operation, e.g. backup-server-01, job Daily-Gold-VMware.
correlationKey	String	applicationId|eventType|resource (D1).
state	Enum	RECEIVED, INVESTIGATING, WAITING_FOR_INFO, COMPLETED, UNABLE_TO_COMPLETE.
resolution	Enum	UNRESOLVED, PARTIALLY_RESOLVED, RESOLVED, UNKNOWN (default UNKNOWN).
stateReason	String	Why the current state was entered (e.g. failure reason).
summary	String	Agent's one-paragraph summary.
eventCount	int	Number of linked events (≥ 1).
recurrenceOfId	String?	Most recent closed investigation with the same key.
toolCallsUsed	int	Counter against the loop cap (D12).
missingInformation	String	What the Agent could not determine, one item per line.
scenarioId	String?	Demo only: which seed data set the simulated adapter uses (A10).

Field	Type	Description
createdAt / firstRecommendationAt / completedAt	Instant	Timestamps; firstRecommendationAt feeds MET-01.

5.2 Child entities

The child entities carry the fields shown in Figure 3. Three of them have rules worth stating:

- **ToolCall**: every tool request is recorded, including rejected ones (status DENIED or ERROR). Its evidenceId (E1, E2, …) is what findings cite.
- **Finding**: `downgraded = true` when the validator demoted a CONFIRMED finding to SUSPECTED (D5).
- **ActionProposal**: stores `paramsHash` and `expiresAt` so execution can prove it matches what was approved (Section 9.4).

6. Investigation state machine

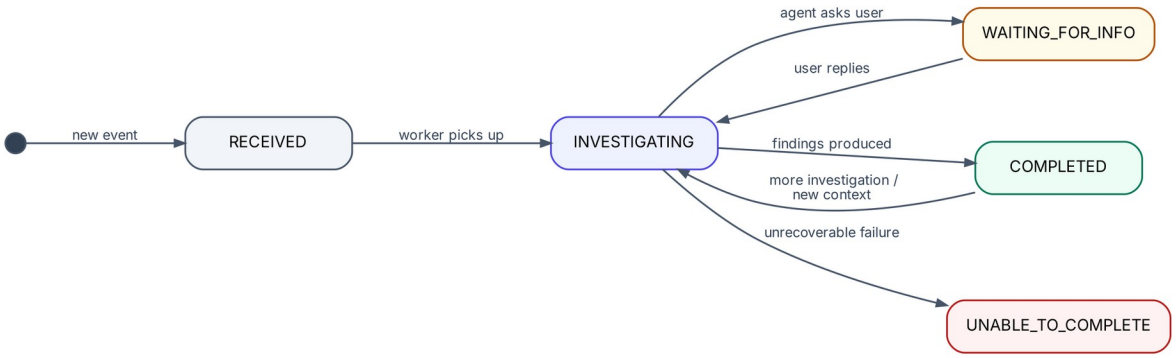


Figure 4. Investigation states and allowed transitions

Table 20. Transition table

From	To	Trigger	Side effects
(none)	RECEIVED	New, non-duplicate event accepted	Investigation created; run queued; audit INVESTIGATION_CREATED.
RECEIVED	INVESTIGATING	Worker starts the run	Progress event published.
INVESTIGATING	WAITING_FOR_INFO	Validated output contains questionForUser	Agent question shown to the user.
WAITING_FOR_INFO	INVESTIGATING	User replies	Reply added as context; run resumes.

From	To	Trigger	Side effects
INVESTIGATING	COMPLETED	Validated final output persisted	Findings, recommendations, optional action proposal saved.
INVESTIGATING	UNABLE_TO_COMPLETE	LLM unavailable after retries, invalid output after repair, or unexpected error	stateReason set; evidence kept.
COMPLETED	INVESTIGATING	User adds context or requests more investigation	Previous findings passed to the new run; pending proposals expire.
UNABLE_TO_COMPLETE	INVESTIGATING	User requests more investigation (retry)	New run with existing evidence.

> Any transition not listed above is rejected with an IllegalStateTransitionException and audited. Plain follow-up questions never change the state (D7).

6.1 Resolution status rules

Table 21. Resolution status rules

Resolution	Set when
UNKNOWN	Default; also when the Agent cannot determine the cause.
UNRESOLVED	Analysis completed and the problem is still present, or an approved action FAILED verification.
PARTIALLY_RESOLVED	Action succeeded and the metric improved but is still beyond its threshold (e.g. S-1C).
RESOLVED	Action succeeded and verification shows the metric within threshold. Only the system sets this value, never the LLM.

7. Event intake and deduplication

7.1 Event schema

```
POST /api/events
X-Api-Key: <application key>
{
  "eventId":     "evt-9f12",              // unique per occurrence
  "application": "backup-server",
  "eventType":   "HIGH_MEMORY",
  "resource":    "backup-server-01",
  "timestamp":   "2026-11-07T10:00:00Z",
  "details":     { "memoryPercent": 93, "threshold": 85 }
}
```

7.2 Validation rules

Table 22. Event validation rules

#	Rule	Failure response
V-E1	API key present and belongs to a configured application.	401 Unauthorized
V-E2	eventId, application, eventType, resource and timestamp present.	400 Bad Request (missing fields listed)
V-E3	Application is configured.	422 UNKNOWN_APPLICATION
V-E4	The API key belongs to the application named in the body.	401 Unauthorized
V-E5	Event type is configured for that application.	422 UNKNOWN_EVENT_TYPE (hint lists valid types)
V-E6	timestamp is ISO-8601 and not more than 24 h in the future.	400 Bad Request

7.3 Intake decision flow

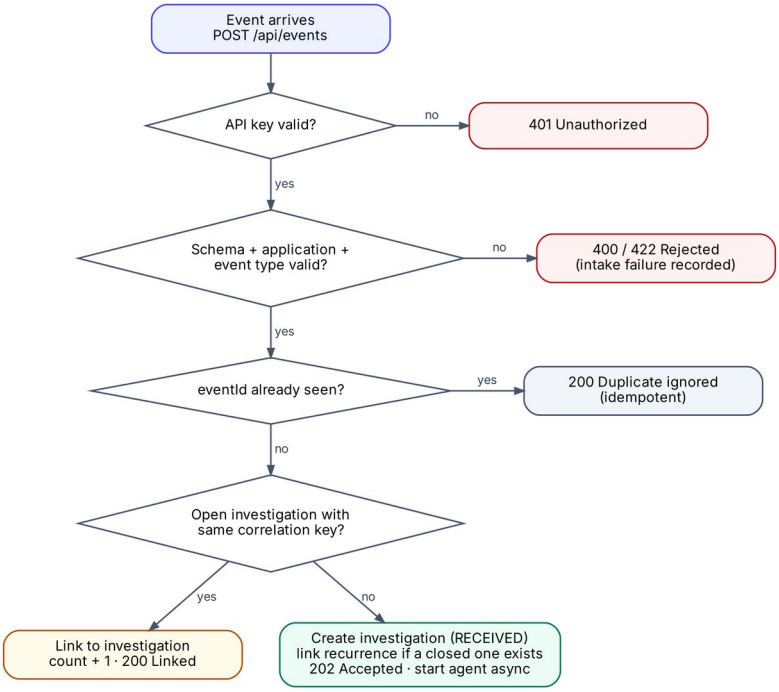


Figure 5. Event intake decision flow

7.4 Correlation and recurrence

Table 23. Correlation outcomes

Situation	Outcome	Response
Same eventId already received	Ignored; nothing changes	200 {status: DUPLICATE_IGNORED, investigationId}
Open investigation (not COMPLETED / UNABLE_TO_COMPLETE) with same key	Event linked; eventCount + 1; event added to timeline	200 {status: LINKED, investigationId}

Situation	Outcome	Response
No open investigation; a closed one exists with same key	New investigation; recurrenceOfId = most recent closed one	202 {status: CREATED, investigationId}
No investigation with same key	New investigation	202 {status: CREATED, investigationId}

7.5 Race safety

Create-or-link runs inside a lock striped by correlation key, in one database transaction. A second thread with the same key waits, then sees the investigation created by the first and links to it. Unit tests fire concurrent identical events and assert that exactly one investigation exists.

7.6 Intake failures

Rejected events (V-E1 to V-E6) are written to an intake-failure log with time, application (if known), reason and a payload excerpt with secrets removed. The Health screen lists the latest 50 (FR-EVT-05, FR-CFG-03).

8. Agent loop and LLM contract

8.1 Loop overview

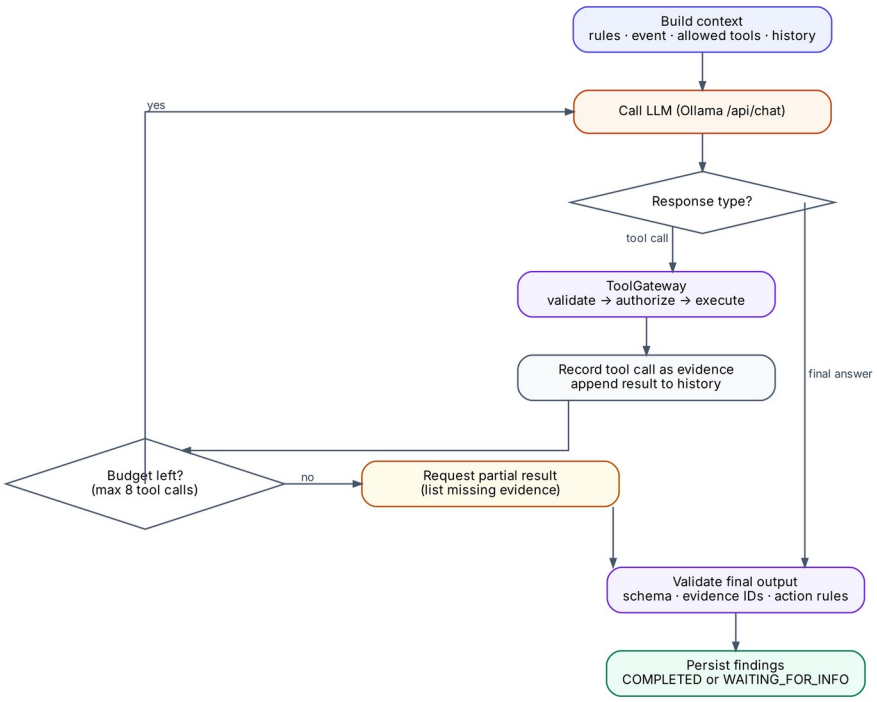


Figure 6. Agent tool-calling loop

A run has two phases (D18). In the **tool phase**, the orchestrator calls Ollama's `/api/chat` endpoint with the conversation so far and the tools allowed for the event type. Each `tool_calls` request goes through the ToolGateway and its result is appended as a tool message. In the **answer phase**, a separate request in JSON mode, without tools, re-lists every evidence item and asks for the final answer.

Guards in code compensate for a small model (D17):

Table 24. Agent guards

Guard	Behavior	Observed problem it fixes
Budget	At most 8 tool calls per run	Endless loops
Nudge	If the model stops before 4 tool calls, it is told which tools it has not used (at most 2 nudges)	Stopping after confirming the symptom
Duplicate call	An identical repeated call is not executed; the model is told to see the earlier evidence ID	Repeating the same call
Use tools before asking	If the model asks the user a question while unused tools and budget remain, it is sent back once to use them	Asking the user for data a tool could provide

8.2 Context sent to the model

Table 25. Model context

Part	Content
Role and rules	You are a support investigator for {application}. Use only the provided tools. Cite evidence IDs. Label each finding CONFIRMED (directly measured), SUSPECTED (inferred) or RULED_OUT. Never claim an action was executed. If information is missing, say what is missing or ask one question. Answer in the JSON schema provided.
Application profile	Name, description, configured thresholds, runbook titles.
Event	Event type, resource, details, event count, recurrence information.
Tools	Only read tools enabled for this application and event type (FR-AI-02). Typically 5–7.
Proposable actions	Names, descriptions and parameters of enabled actions. Listed as text, not as callable tools (P5).
Conversation	Prior tool results (with evidence IDs), user context and replies, prior findings when re-investigating.
Final instruction	All evidence re-listed, the JSON format, and decision rules: list abnormal observations first, CONFIRMED only for directly measured facts, propose an action only if it fixes a confirmed problem.

> The exact prompt wording is iterated during implementation and recorded in the AI usage log. This section fixes what must be in the prompt, not the final wording.

8.3 Tool results as evidence

- Each executed or rejected tool call is stored as a ToolCall with the next evidence ID (E1, E2, …).
- The result is capped before it is sent to the model: at most 1,500 characters, with a truncation note. Logs are filtered to matching lines by the adapter.
- Rejected calls are returned to the model as structured errors (e.g. `INVALID_ARGUMENT: hours must be 1–168`) so it can correct itself.

8.4 Final output schema

```
{
  "observations": [{"evidenceId": "E3", "fact": "1172 stale sessions, 8 running jobs"}],
  "summary": "Backup Server memory has risen for 6 days ...",
  "findings": [
    {"kind": "CONFIRMED", "statement": "Heap usage is 93% (threshold 85%)",
"evidenceIds": ["E1"]},
    {"kind": "SUSPECTED", "statement": "Stale sessions are not released ...",
"evidenceIds": ["E3","E4"]},
    {"kind": "RULED_OUT", "statement": "No abnormal job volume", "evidenceIds": ["E3"]}
  ],
  "missingInformation": ["Thread state of session-cleanup before the upgrade"],
  "recommendations": [{"text": "Escalate thread dump to engineering", "priority": 2}],
  "proposedAction": {"name": "purge_stale_sessions", "params": {}, "reason": "..."},
  "questionForUser": null,
  "resolution": "UNRESOLVED"
}
```

8.5 Output validation rules

The ResponseValidator applies these rules before anything is persisted. They are the deterministic guardrails over AI output (P1).

Table 26. Response validation rules

#	Rule	On violation	Req.
R-V1	Output parses against the schema.	One repair retry with the parse error; then UNABLE_TO_COMPLETE.	NFR-REL-01
R-V2	Every cited evidence ID exists in this investigation.	Unknown IDs removed.	FR-DATA-03
R-V3	A CONFIRMED finding cites at least one valid evidence ID with status OK.	Downgraded to SUSPECTED; downgraded = true; shown in UI.	FR-AI-03
R-V4	proposedAction exists, is enabled and belongs to this application.	Proposal dropped; a MANUAL recommendation with the action's instructions is added.	FR-ACT-06
R-V5	At least one finding, or a non-empty missingInformation.	missingInformation set to “Cause could not be determined”; resolution UNKNOWN.	FR-AI-04
R-V6	resolution is UNRESOLVED or UNKNOWN.	Any other value replaced by UNKNOWN (only the system sets RESOLVED).	D4, D8
R-V7	questionForUser present.	State WAITING_FOR_INFO instead of COMPLETED.	FR-DATA-04
R-V8	A proposal is kept only when resolution is UNRESOLVED.	Proposal dropped; a note explains why (D19).	FR-ACT-03

8.6 Limits and parameters

Table 27. Agent limits

Parameter	Value	Configurable
Model	qwen2.5:3b	Yes

Parameter	Value	Configurable
Minimum tool calls before nudging	4 (max 2 nudges)	Yes
Temperature	0.1	Yes
Max tool calls per run	8	Yes
LLM call timeout	60 s	Yes
LLM retries	2, exponential backoff (1 s, 3 s)	Yes
Tool execution timeout	5 s	Yes
Max tool result sent to model	1,500 characters	Yes

8.7 Follow-ups, context and re-investigation

Table 28. Follow-up handling

User input	Behavior	State change
Question (“Will this come back?”)	Single LLM call with the summary, findings and evidence; answer added to the conversation. No tools.	None
Reply to the Agent's question	Reply stored as context; run resumes.	WAITING_FOR_INFO → INVESTIGATING
Additional context (“we patched last night”)	Context stored; new run with prior findings.	COMPLETED → INVESTIGATING
“Investigate more”	New run with prior findings and remaining budget reset.	COMPLETED / UNABLE_TO_COMPLETE → INVESTIGATING

8.8 Untrusted content

Log lines, tool results and user messages are data, not instructions. Even if such content tries to steer the model, the model can only call read-only tools through the gateway, and any state change still needs a human approval with role checks. The worst outcome of manipulated content is a misleading text, which is mitigated by evidence citation and labeling.

9. Tools, permissions and approval gate

9.1 Tool and action definitions

Table 29. Definition fields

Field	Meaning
name	Unique within the application, e.g. get_job_sessions.
kind	READ (callable by the LLM) or ACTION (proposable only, needs approval).
description	Shown to the LLM; written for the model.
parameters	JSON schema: type, enum, min/max, default.
enabled	Runtime toggle controlled by ADMIN.
eventTypes	READ tools: event types for which the tool is offered.
risk / requiredRole	ACTION only: LOW / HIGH and the minimum role to approve (D3).

Field	Meaning
verify	ACTION only: read tool, optional fixed arguments, a condition for RESOLVED and an optional partial condition for PARTIALLY_RESOLVED (D8).
instructions	ACTION only: manual steps shown when the action is not available (FR-ACT-06).

9.2 ToolGateway pipeline

Every tool or action execution, from the agent loop or from an approval, passes through the same ordered checks.

Table 30. Gateway checks

Step	Check	Error code returned
G1	Tool is defined for this investigation's application.	UNKNOWN_TOOL
G2	Tool is enabled (admin toggle).	TOOL_DISABLED
G3	From the agent loop, kind must be READ. ACTION is only reachable from ApprovalService.	ACTION_NOT_CALLABLE
G4	Arguments validated against the schema. Safe coercion allowed ("7" → 7); out-of-range values rejected; missing optional values defaulted.	INVALID_ARGUMENT
G5	Adapter executes with a 5 s timeout.	TIMEOUT / ADAPTER_ERROR
G6	Request and outcome recorded as ToolCall + AuditEntry, including denials.	–

> Motivation from the first model spike: llama3.2:3b called get_metrics with {"days": "0"}, a string with a meaningless value. G4 exists for exactly this.

9.3 Role permissions

Table 31. Role permission matrix

Capability	APP_USER	SUPPORT_ENGINEER	ADMIN
View investigations, evidence, history	✓	✓	✓
Ask questions, add context, give feedback, request more investigation	✓	✓	✓
Send simulated events (Event Simulator)	✓	✓	✓
Approve or reject actions	if action requires APP_USER	if requiredRole ≤ SUPPORT_ENGINEER	all
View Health and Metrics		✓	✓
Change application configuration and toggles			✓

Roles are ordered APP_USER < SUPPORT_ENGINEER < ADMIN. Checks are enforced on the server in two layers: URL rules and method-level checks in services. Hidden buttons are a usability aid, not a control.

9.4 Approval gate

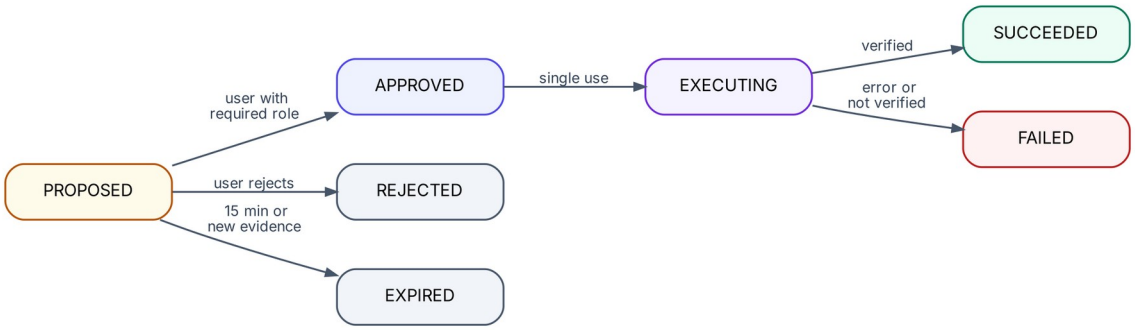


Figure 7. Action proposal lifecycle

Table 32. Approval rules

#	Rule	Req.
AR1	The approval card shows the action, target, exact parameters, expected effect and risk.	NFR-USAB-01
AR2	Approval is single-use: APPROVED moves immediately to EXECUTING and cannot be replayed.	FR-ACT-04
AR3	A proposal expires after 15 minutes, or when new evidence or a new run makes it stale.	FR-ACT-03
AR4	Users may reject a proposal with an optional reason.	FR-ACT-07
AR5	Execution uses the stored parameters; a hash check guarantees they are unchanged.	FR-ACT-04
AR6	Every decision and outcome is audited with user and time.	FR-ACT-07
AR7	Users below requiredRole see “Requires {role} approval”; the server rejects their approve call with 403.	NFR-SEC-02

9.5 Execution and verification

1. ApprovalService checks role, status PROPOSED, expiry and parameter hash.
2. The action runs through the ToolGateway (G1, G2, G4–G6) with the stored parameters.
3. The verify read tool runs and its conditions are evaluated, e.g. `current < thresholds.memoryPercent`. Condition true: RESOLVED. Otherwise partial condition true: PARTIALLY_RESOLVED. Otherwise FAILED.
4. Outcome SUCCEEDED only if execution returned OK and verification passed; otherwise FAILED with the reason. A FAILED action is never shown as successful (FR-ACT-05).
5. Resolution is updated according to Section 6.1 and the result is posted to the conversation.

10. Applications and scenarios

10.1 Applications

Table 33. Applications

Application ID	Description	Event types
backup-server	SafeKeep Backup Server: management, scheduling, job sessions, reporting, datasources	HIGH_MEMORY HIGH_CPU JOB_FAILED REPORT_JOB_FAILED DATASOURCE_UNHEALTHY DEPENDENCY_UNAVAILABLE
storage-server	SafeKeep Backup Storage Server: disk pools, expiration, replication	HIGH_STORAGE_USAGE REPLICATION_JOB_FAILED UNEXPECTED_RESTART

Each application's API key is read from an environment variable named in its configuration (`BACKUP_SERVER_API_KEY`, `STORAGE_SERVER_API_KEY`).

10.2 Application configuration (example)

```
safekeep:
  applications:
    storage-server:
      name: SafeKeep Backup Storage Server
      thresholds: { poolUsagePercent: 90, memoryPercent: 85 }
      event-types: [HIGH_STORAGE_USAGE, REPLICATION_JOB_FAILED, UNEXPECTED_RESTART]
      tools:
        - name: get_pool_status
          description: Usage of the storage pool, capacity and daily growth.
          event-types: [HIGH_STORAGE_USAGE, REPLICATION_JOB_FAILED]
        - name: get_system_logs
          event-types: ["*"]
          parameters:
            - { name: boot, type: string, allowed: [current, previous], default-value:
current }
      actions:
        - name: run_expiration_cleanup
          risk: HIGH
          required-role: ADMIN
          verify:
            tool: get_pool_status
            condition: "usagePercent < thresholds.poolUsagePercent"
            partial-condition: "usagePercent < 98"
          instructions: In Storage > Maintenance, start Expiration.
      runbooks:
        - runbooks/storage-server/storage-pool-full.md
```

API keys are bound separately from environment variables (`safekeep.api-keys.storage-server: ${STORAGE_SERVER_API_KEY:}`).

10.3 Tool catalog

Table 34. Read tools (each event type exposes a subset of 5–7)

Application	Tool	Returns	Main use
backup-server	get_metrics(metric: cpu|memory|login_rate|auth_sessions, hours)	Time series summary	UC-1
backup-server	get_top_processes(sortBy: cpu|memory, limit)	Ranked processes	UC-1
backup-server	get_job_sessions(status)	Session counts and samples	S-1A
backup-server	get_job_details(jobId)	Job, per-VM results, errors	UC-2
backup-server	get_logs(source, level, contains, limit)	Matching log lines	All
backup-server	get_recent_changes(days)	Upgrades, setting changes	UC-1, UC-2
backup-server	get_settings(section)	Configuration values	UC-2
backup-server	get_storage_targets()	Capacity of connected storage	S-2A, S-3B
backup-server	get_dependency_status(name)	Reachability, TLS, last success	UC-3
backup-server	get_affected_operations(dependency)	Operations depending on it	UC-3
both	search_runbook(query)	Matching runbook excerpts	All
storage-server	get_pool_status()	Usage %, growth trend	S-1C
storage-server	get_expiration_status()	Last run, pending expirations	S-1C
storage-server	get_replication_status(jobId)	Throughput, errors	S-2E
storage-server	get_system_logs(boot)	Kernel/system log lines	S-4A
storage-server	get_disk_health()	Per-disk I/O error counts	S-4A

Table 35. Actions

Application	Action	Risk	Required role	Verification
backup-server	purge_stale_sessions	LOW	SUPPORT_ENGINEER	stale sessions = 0, memory below threshold
backup-server	clear_expired_auth_sessions	LOW	SUPPORT_ENGINEER	auth sessions and CPU drop
backup-server	retry_failed_vms(jobId)	LOW	SUPPORT_ENGINEER	job status SUCCESS
backup-server	retry_report_job(jobId)	LOW	APP_USER	report status SUCCESS
storage-server	run_expiration_cleanup	HIGH	ADMIN	pool usage below threshold
storage-server	retry_replication(jobId)	LOW	SUPPORT_ENGINEER	replication status SUCCESS

10.4 Seed data rules

- Seed data holds raw observations only: numbers, log lines, records. It never contains a root-cause field (A5).
- Every scenario includes normal-looking noise (e.g. healthy storage in S-2B) so the Agent must rule causes out.
- Format: `seed/<app>/baseline.json` holds normal values; `seed/<app>/<scenario>.json` overrides them. Each tool maps selectors such as `metric=memory` (or `*`) to a result.

- Actions may declare `effects` that change later tool results, so outcomes are observable (e.g. memory 93% → 61%). `_error` / `error` entries simulate failures.
- Each scenario can be reset from the simulator, restoring its initial state.

10.5 Scenario catalog

Table 36. Scenario catalog (8 core, 3 extra)

ID	App / event	Root cause in data	Expected Agent outcome	Tier
S-1A	backup-server HIGH_MEMORY	1,172 stale job sessions; cleanup task stuck since an upgrade 6 days ago	Confirmed stale sessions; suspected stuck cleanup; propose purge → SUCCEEDED → RESOLVED	Core
S-1B	backup-server HIGH_CPU	Login rate 1,200/min (baseline 15); 48,000 auth sessions: new session per job step	Suspected missing token reuse; clear sessions → SUCCEEDED; recurrence warning; escalate	Core
S-1C	storage-server HIGH_STORAGE_USAGE	Pool 98%; expiration not run for 9 days	Propose cleanup (ADMIN) → SUCCEEDED → 92% → PARTIALLY_RESOLVED	Core
S-2A	backup-server JOB_FAILED	6 of 40 VMs failed with “insufficient space”; storage 98%	Retry approved → FAILED (still full) → UNRESOLVED; manual steps	Core
S-2B	backup-server JOB_FAILED	API timeouts in job logs; CPU 96%; auth loop; storage healthy	Rules out storage; links cause to auth loop	Extra (designed, not seeded)
S-2C	backup-server JOB_FAILED	Generic error, sparse logs, all metrics normal	Cause unknown; states missing info; asks user → WAITING_FOR_INFO	Core
S-2D	backup-server REPORT_JOB_FAILED	Report query timeout: 365 days × 5,000 VMs	Suggest narrower range; retry (APP_USER can approve)	Extra
S-2E	storage-server REPLICATION_JOB_FAILED	Replication throughput dropped to 5% of normal	Network suspected; retry; escalate to network team	Extra
S-3A	backup-server DATASOURCE_UNHEALTHY	vCenter TLS certificate expired yesterday	Lists affected / unaffected operations; no action permitted; escalation steps	Core
S-3B	backup-server DEPENDENCY_UNAVAILABLE	storage-01 unreachable since 02:14	Backups to it queued, restores unavailable; links to S-4A	Core
S-4A	storage-server UNEXPECTED_RESTART	Previous-boot log: kernel panic after repeated I/O errors on disk sdc	Hardware cause; no action; replace disk, run pool health check	Core

10.6 Adding a new application

1. Create `apps/<id>.yaml` with event types, tools, actions, thresholds and runbooks.
2. Implement `AppAdapter` for the application (tools and actions).
3. Provide seed data or, for a real system, an HTTP-backed adapter.

No change is needed in intake, orchestrator, gateway, approval, UI or persistence (FR-CFG-04, NFR-MAINT-01). Unit tests register a third test-only application to prove this.

11. REST API

Table 37. REST endpoints

Method	Path	Auth	Purpose	Req.
POST	/api/events	API key	Submit an event	FR-EVT-01…05
GET	/api/investigations?state=&app=&from=&to=	Session or Basic	List / history	FR-CFG-02
GET	/api/investigations/{id}	Session or Basic	Full detail	FR-USR-01
GET	/api/investigations/{id}/stream	Session or Basic	Live progress (SSE)	FR-INV-03
POST	/api/investigations/{id}/messages	Session or Basic	Question, context or reply	FR-USR-03, -04
POST	/api/investigations/{id}/reinvestigate	Session or Basic	Request more investigation	FR-USR-05
POST	/api/investigations/{id}/proposals/{pid}/approve	Session or Basic + role	Approve action	FR-ACT-03
POST	/api/investigations/{id}/proposals/{pid}/reject	Session or Basic + role	Reject action	FR-ACT-07
POST	/api/investigations/{id}/feedback	Session or Basic	Useful / not useful	FR-USR-06
GET	/api/applications	ADMIN	Configuration view	FR-CFG-01
PATCH	/api/applications/{app}/tools/{name}	ADMIN	Enable / disable tool or action	FR-ACT-01
GET	/api/health	SUPPORT+	Ollama, adapters, intake errors	FR-CFG-03
GET	/api/metrics	SUPPORT+	MET-01…03	MET
GET	/api/simulator/scenarios	Session or Basic	Scenario catalog	Demo
POST	/api/simulator/send	Session or Basic	Send a scenario event through the same intake path with the app key	Demo
POST	/api/simulator/reset	Session or Basic	Reset scenario state	Demo

All authenticated endpoints accept a browser session or HTTP Basic credentials (D20). Errors use one format, so the UI can show actionable messages (NFR-USAB-01):

```
{
  "error":   "UNKNOWN_EVENT_TYPE",
  "message": "Event type DISK_FULL is not configured for storage-server",
  "hint":    "Configured: HIGH_STORAGE_USAGE, REPLICATION_JOB_FAILED, ..."
}
```

12. User interface

12.1 Design principles

- Calm, information-dense operations-tool look: neutral palette with one indigo accent, Inter typeface, cards with soft borders, light and dark themes.
- Color carries meaning only: state badges, and finding labels (CONFIRMED green, SUSPECTED amber, RULED_OUT gray).
- Progress is shown as a live vertical timeline of agent steps.
- Approval is a distinct card that states exactly what will change.

12.2 Screen map

Table 38. Screens

Screen	Roles	Purpose
Login	All	Sign in; quick-select buttons for the three demo users.
Event Simulator	All	Choose application and scenario; preview event JSON; send; reset scenario.
Investigations	All	List and history with filters (state, application, date).
Investigation detail	All	The working screen (12.4).
Applications	ADMIN	Per-application event types, tools and actions with toggles.
Health & Metrics	SUPPORT_ENGINEER, ADMIN	Agent health, connection status, intake errors, MET-01…03.

12.3 Investigations list

Columns: ID, application, event type, resource, state badge, resolution badge, event count (with “recurrence” marker), created, last updated. Rows update live. Filters make this screen the investigation history (FR-CFG-02).

12.4 Investigation detail

Table 39. Investigation detail areas

Area	Content	Req.
Header	Event summary, application, resource, state and resolution badges, linked events count, recurrence link.	FR-USR-01
Timeline	Live steps: received, each tool call (name, arguments, status), analysis, completion.	FR-INV-03
Findings	Grouped CONFIRMED / SUSPECTED / RULED_OUT; each cites evidence chips (E3). “Downgraded by validator” marker where applicable.	FR-AI-03
Evidence	Expandable tool results per evidence ID.	FR-DATA-03
Missing information	What could not be determined and what to check next.	FR-AI-04
Recommendations	Ordered next steps; MANUAL steps include instructions.	FR-ACT-06

Area	Content	Req.
Approval card	Action, target, parameters, expected effect, risk, required role, expiry countdown, Approve / Reject.	FR-ACT-03
Conversation	Agent questions, user questions and answers; input to ask, add context, or request more investigation.	FR-USR-03…05
Feedback	Useful / not useful with optional comment.	FR-USR-06
Audit	Collapsible chronological audit trail.	FR-ACT-07

12.5 Applications (admin)

For each application: connection status, event types, thresholds, read tools and actions with enable/disable toggles, risk and required role. Changes take effect immediately for new tool calls and are audited.

12.6 Health and metrics

Table 40. Health and metrics panels

Panel	Content
Agent health	Worker pool usage, queued runs, investigations by state.
LLM	Ollama reachable and model downloaded (UP / DEGRADED / DOWN).
Applications	Adapter status per application, last event received.
Intake errors	Latest 50 rejected events with reasons.
MET-01	Median time from event to first recommendation.
MET-02	% investigations with evidence-backed findings; % positive feedback.
MET-03	Approved action success rate; actions executed without approval (always 0).

13. Error handling and reliability

13.1 Failure modes

Table 41. Failure handling

Failure	Behavior	User sees	Req.
Invalid / unauthorized event	Rejected; intake failure logged	Health: intake errors	FR-EVT-05
Tool argument invalid	Not executed; error returned to model to self-correct	Timeline step marked rejected	FR-ACT-02
Tool disabled by admin	Denied (TOOL_DISABLED); model continues with other tools	Timeline step marked denied	FR-DATA-02
Adapter error / timeout	Recorded; returned to model as error	Timeline step marked failed	NFR-REL-01
Ollama unreachable	2 retries; then UNABLE_TO_COMPLETE	“AI service unavailable”; evidence kept; retry button	NFR-REL-01
Malformed model output	One repair retry; then UNABLE_TO_COMPLETE	Reason shown	NFR-REL-01

Failure	Behavior	User sees	Req.
Tool budget exhausted	Partial result requested	Findings plus missing information	D12
Action execution or verification fails	Proposal FAILED; resolution UNRESOLVED	Failure and reason in conversation	FR-ACT-05
Application restart during a run	On startup, INVESTIGATING runs become UNABLE_TO_COMPLETE (“interrupted”); RECEIVED runs are resubmitted	Retry available	NFR-REL-01
Worker queue full	Investigation stays RECEIVED; retried every 15 s	Investigation appears as received	NFR-REL-01

13.2 Isolation between investigations

Isolation follows from the concurrency model in Section 4.6. A unit test runs two investigations concurrently with a fake LLM and asserts that their evidence and findings never mix (NFR-SCAL-01).

14. Security

Table 42. Security controls

Concern	Control	Req.
User authentication	Spring Security form login, BCrypt-hashed demo passwords.	NFR-SEC-02
Authorization	URL rules by role, plus checks inside services: the approval gate compares the user's role with the action's requiredRole.	NFR-SEC-02
Application authentication	Per-application API key from environment variables; constant-time comparison.	NFR-SEC-01
Secret handling	Keys never logged, rendered, stored in seed data, or sent to the LLM; log masking filter.	NFR-SEC-01
CSRF	Enabled for browser sessions (token sent by the UI). Exempt: the event API (API key) and requests with an Authorization header (D20).	NFR-SEC-02
Input validation	Explicit event field checks (V-E1 to V-E6); schema validation for tool arguments.	FR-ACT-02
Model privileges	Read-only tools only; actions need human approval (P5).	FR-ACT-03
Auditability	Append-only audit of tool calls, approvals, rejections, outcomes, config changes.	FR-ACT-07
Data locality	All processing local (Ollama); no data leaves the machine.	NFR-SEC-01

15. Out of scope and future work

Table 43. Out of scope

Item	Why out of scope	Future direction
Real application integrations	No access to real systems; simulated adapters prove the contract	HTTP adapters per product API
Enterprise SSO and user management	Three seeded users demonstrate enforcement (D2)	OIDC / SAML, roles from directory groups

Item	Why out of scope	Future direction
Persistent admin toggles	In memory for the demo (A9)	Store in the database
Per-application access control	Roles are global (A4)	Role bindings per application
Cross-event-type correlation	Correlation key is per event type (D1)	Time-window clustering of related events
Semantic runbook search	Keyword search sufficient	Embeddings + vector store
Distributed scaling	Single node (A3)	Message queue + distributed locks
Notifications	Not required by PRD	Email / chat notifications
Secret vault	Environment variables sufficient for demo	HashiCorp Vault or cloud KMS

16. Revision history

Table 44. Revision history

Revision	Changes
1	Initial specification, written before implementation.
2	Updated after implementation and testing: Spring Boot 4.1.2 (D16); model qwen2.5:3b with the measured comparison (D15); agent guards (D17); two-phase answer with evidence re-list and observations (D18); rule R-V8 (D19); CSRF and HTTP Basic for API clients (D20); context-window check (D21); intake check order (Section 7.2); verification arguments and partial condition (Section 9); seed data format and demo scenario header (Section 10, A10); persistence and restart behavior (A9, Section 13); full-queue retry (Section 4.6); UI built with plain JavaScript instead of htmx.